

Artificial Intelligence - Deep Learning fundamentals

Master in Artificial Intelligence & Computational Engineering
Luis Montesano



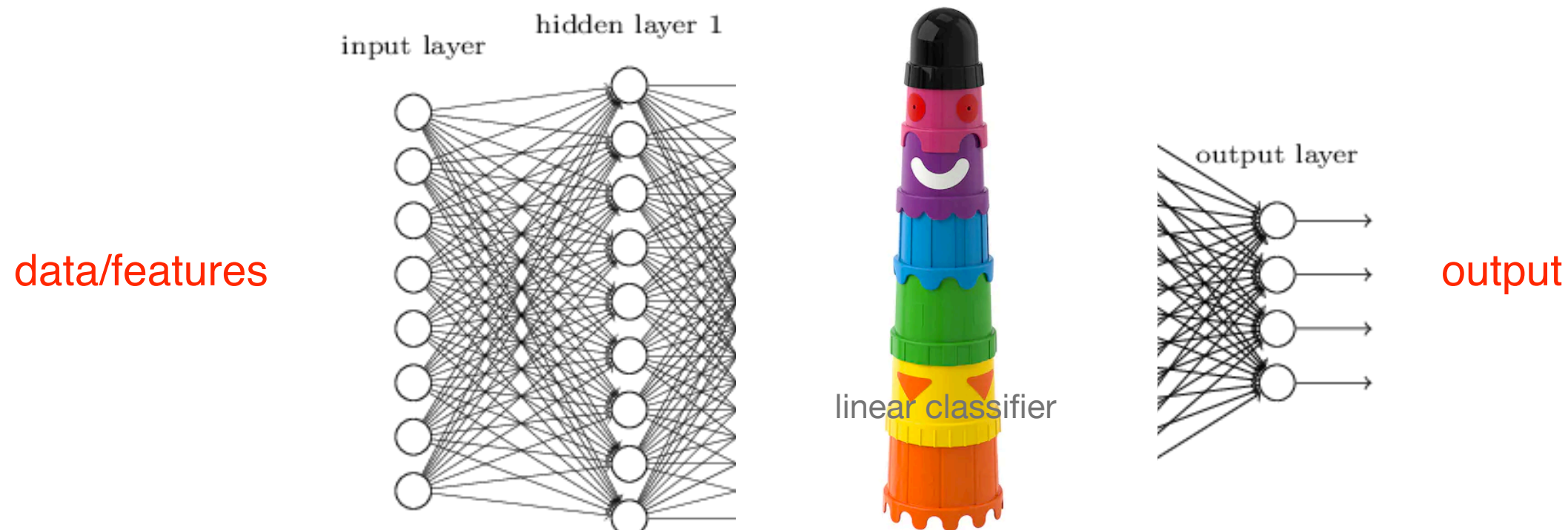
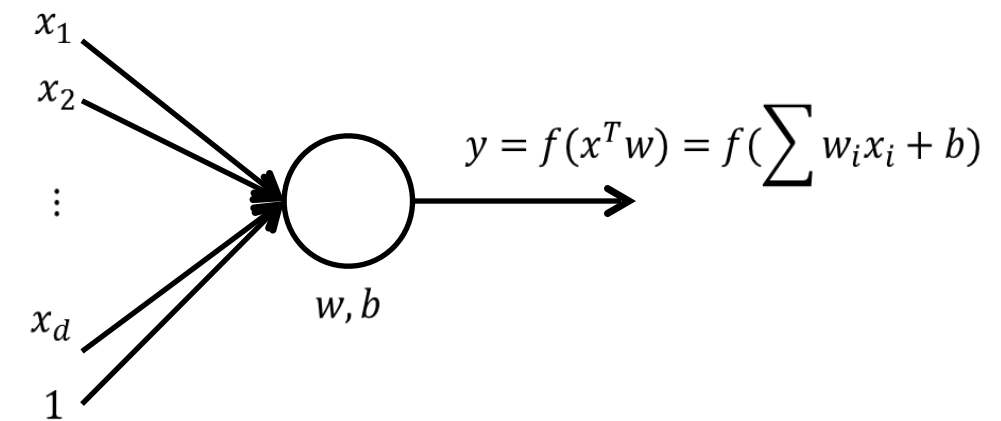
Universidad
Zaragoza

-
- What's deep learning?
 - Related topics
 - DL pipeline
 - **Fundamentals of DL**
 - **Review basic concepts**
 - NN and DNN

Neural Networks and Deep Neural Networks

- **Neural Networks**

- **Neuron:** atomic computational unit in NN.
Params: w , b and f .
- Activation function (f) usually non linear
- Neural Network: connect several layers, neurons, ...
- *Perceptron*: simplest NN. 1 layer - binary linear classifier



Neural Networks and Deep Neural Networks

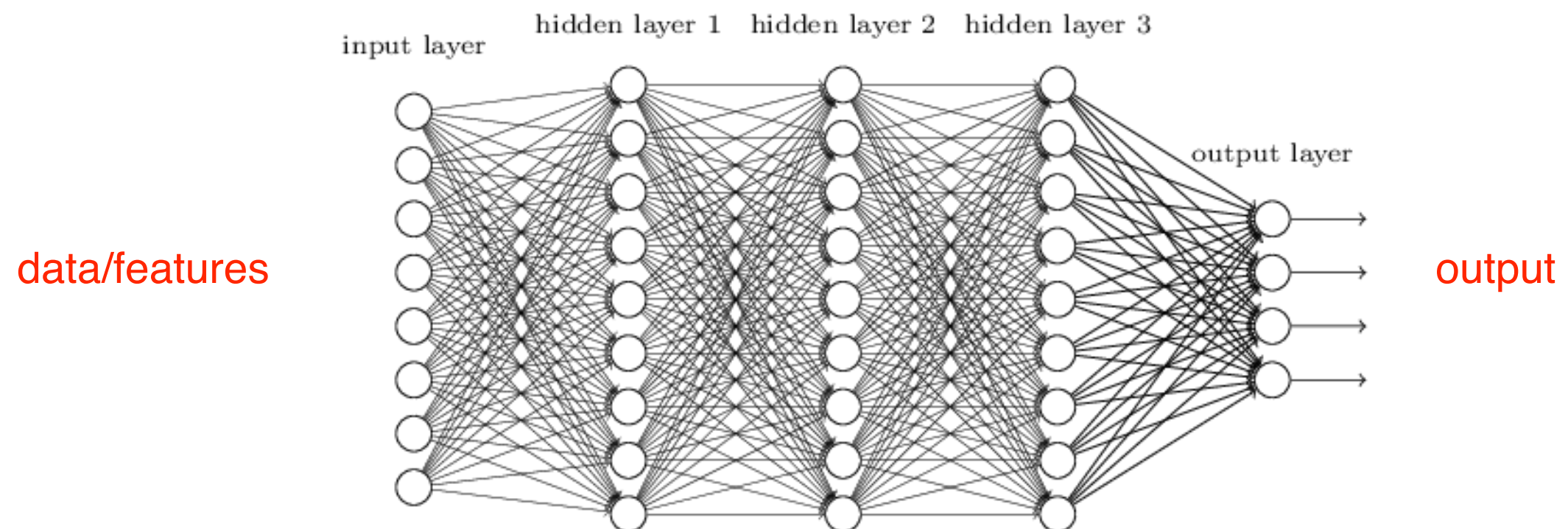
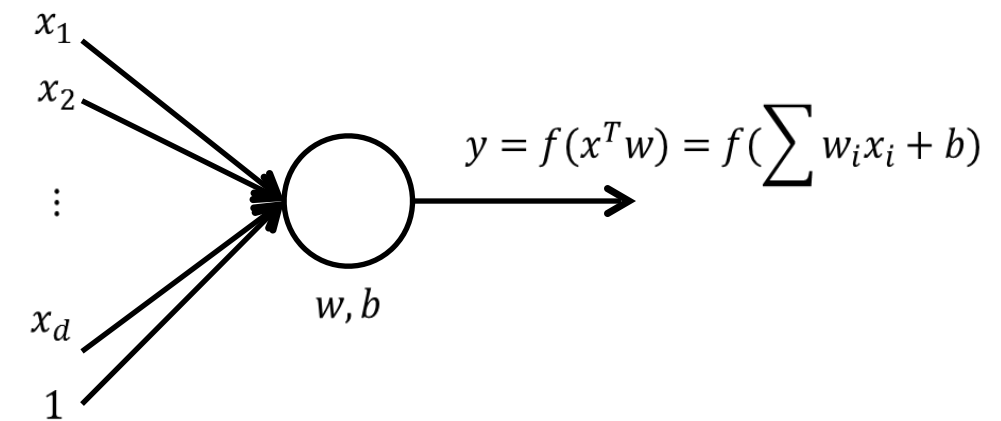
- **Neural Networks**

- Neuron: atomic computational unit in NN.
Params: w , b and f .

- Activation function (f) usually non linear

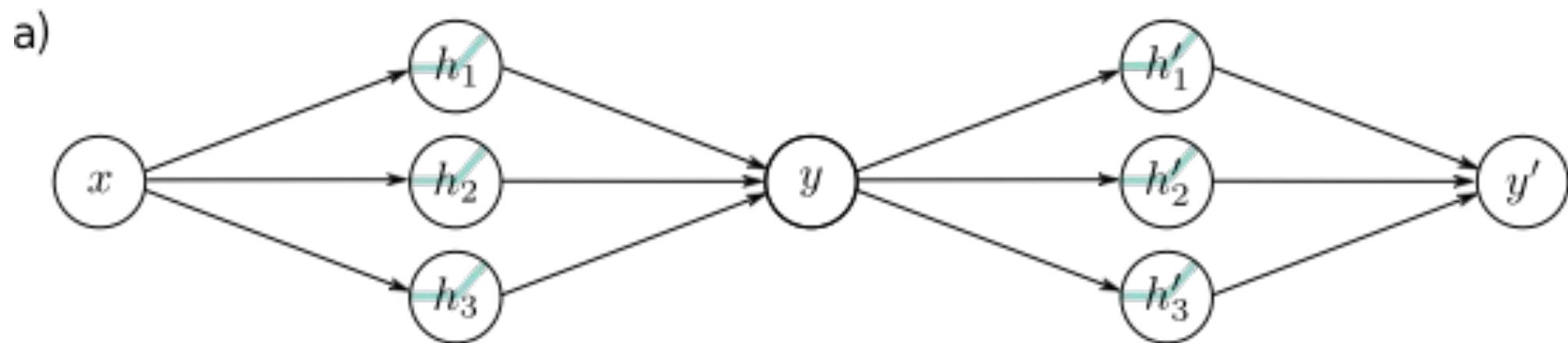
- Neural Network: connect several layers, neurons, ...

- **Deep** Neural Network: many hidden layers

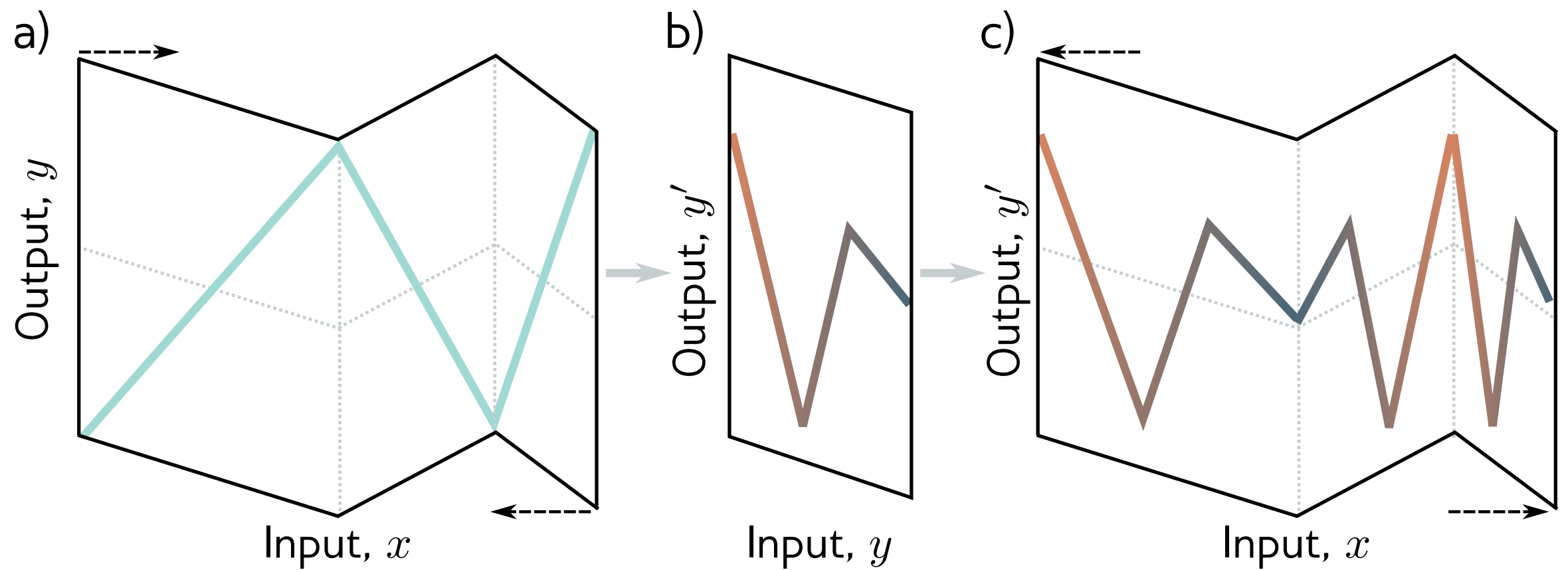


Two layers

- What happens if we stack two layers in 1D?



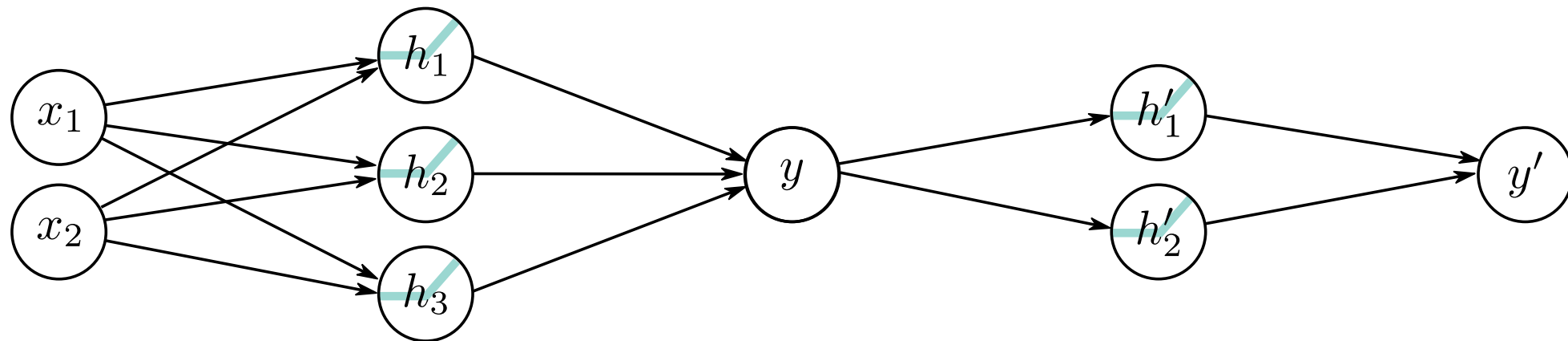
Some intuition of the composition



Two layers

- What happens if we stack two layers in 2D?

a)



Deep network architecture

- K layers = depth of network
- D_k hidden units per layer = width of network
- These are called hyperparameters – chosen before training the network
- Can try retraining with different hyperparameters – hyperparameter optimization or hyperparameter search

General formulation

$$\mathbf{h}_1 = \mathbf{a}[\boldsymbol{\beta}_0 + \boldsymbol{\Omega}_0 \mathbf{x}]$$

$$\mathbf{h}_2 = \mathbf{a}[\boldsymbol{\beta}_1 + \boldsymbol{\Omega}_1 \mathbf{h}_1]$$

$$\mathbf{h}_3 = \mathbf{a}[\boldsymbol{\beta}_2 + \boldsymbol{\Omega}_2 \mathbf{h}_2]$$

$$\vdots$$

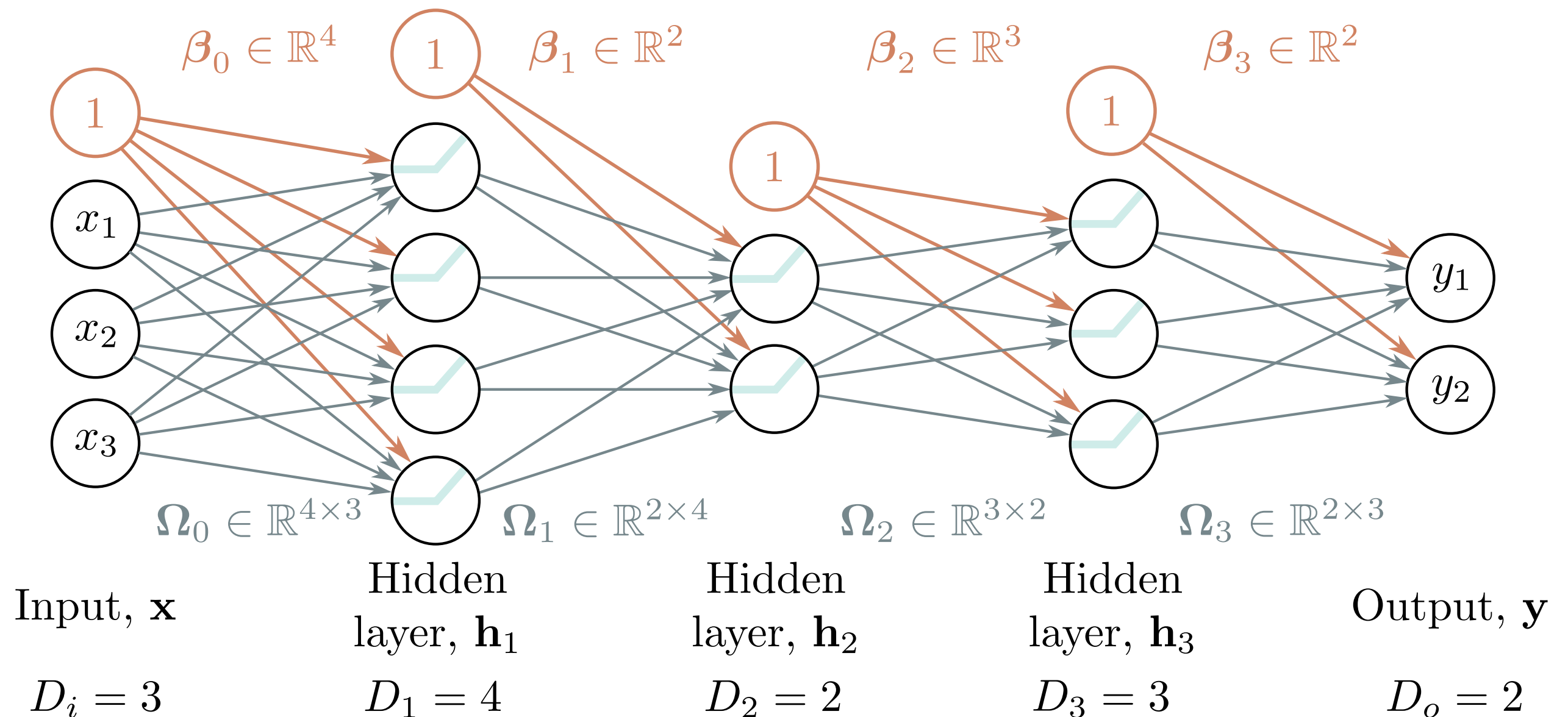
$$\mathbf{h}_K = \mathbf{a}[\boldsymbol{\beta}_{K-1} + \boldsymbol{\Omega}_{K-1} \mathbf{h}_{K-1}]$$

$$\mathbf{y} = \boldsymbol{\beta}_K + \boldsymbol{\Omega}_K \mathbf{h}_K,$$

$$\mathbf{y} = \boldsymbol{\beta}_K + \boldsymbol{\Omega}_K \mathbf{a} [\boldsymbol{\beta}_{K-1} + \boldsymbol{\Omega}_{K-1} \mathbf{a} [\dots \boldsymbol{\beta}_2 + \boldsymbol{\Omega}_2 \mathbf{a} [\boldsymbol{\beta}_1 + \boldsymbol{\Omega}_1 \mathbf{a} [\boldsymbol{\beta}_0 + \boldsymbol{\Omega}_0 \mathbf{x}]] \dots]]]$$

- Bias vectors and weight vectors

Three hidden layers example



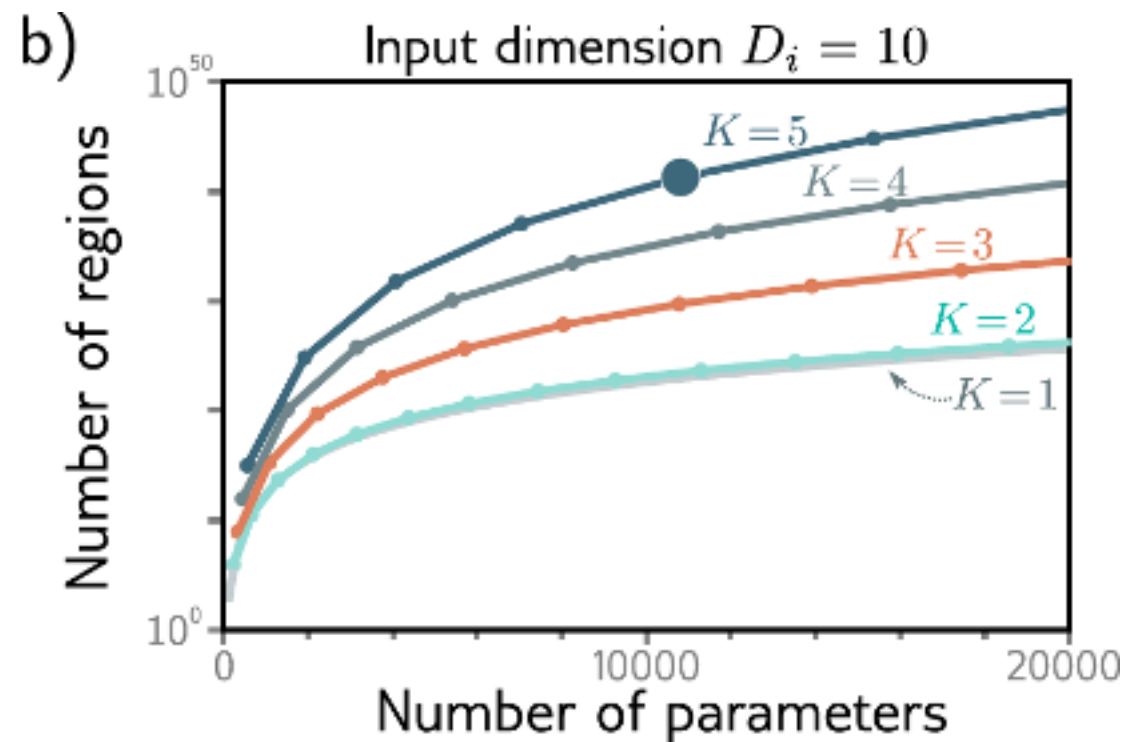
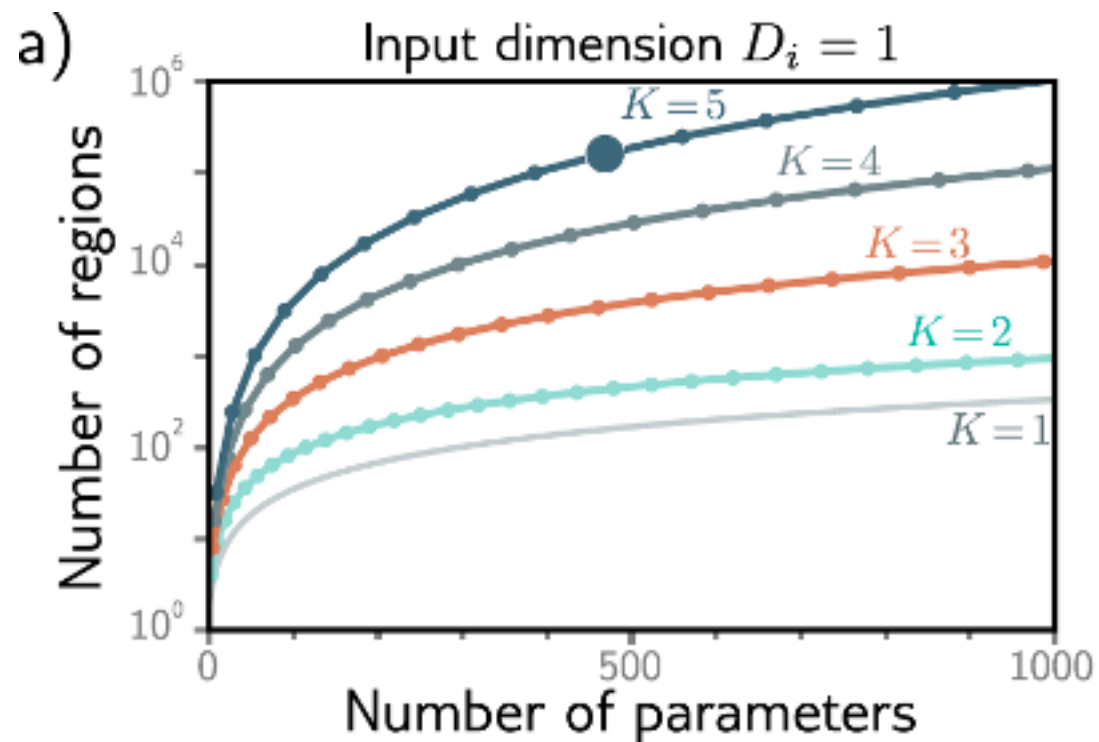
Deep Neural Network

- Deep **feed-forward** Neural Networks:
data propagated through the network to predict the output
 - SoA methods for almost any domain
 - Best results with +50 up to thousand layers
- Deep architectures:

*What do you think are **good/bad properties**?*

Shallow vs Deep Networks (I)

- Representation capacity?
- Number of regions per parameter?
 - Deep networks create many more regions per parameters
 - But there are dependencies between them



Shallow vs Deep Networks (II)

- Depth efficiency
 - Some functions require exponentially more hidden units than a dn for an equivalent approximation
 - There may exist also the same result for shallow networks
 - What functions are in the real world that we want to approximate?
- Structured networks for structured representations
- Fitting and generalisation

Deep Neural Network - Summary

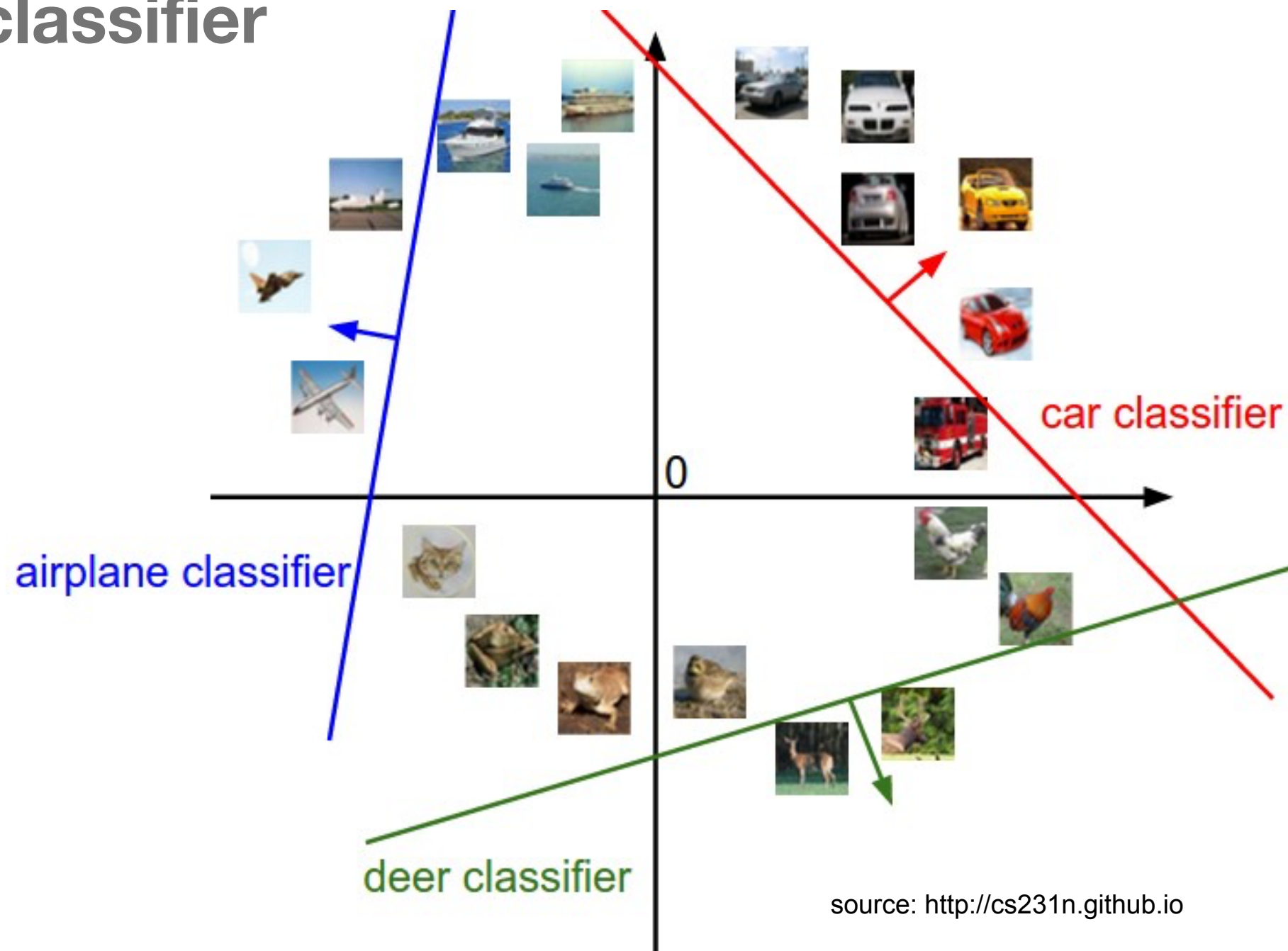
- Deep **feed-forward** Neural Networks:
data propagated through the network to predict the output
- Deep architectures:
 - are more **efficient** than shallow and more **powerful** (model non-linearity more easily)
 - are a good model for **hierarchical data** (images and language among others).
 - work well in practice
 - are still **difficult to design and train**, not always fully understood
 - usually require **a lot of infrastructure**
 - ...

Question

- We have now a new family of models
- Next steps?

Fundamentals of DL

Many basic concepts should be familiar, related to a simple **linear classifier**



source: <http://cs231n.github.io>

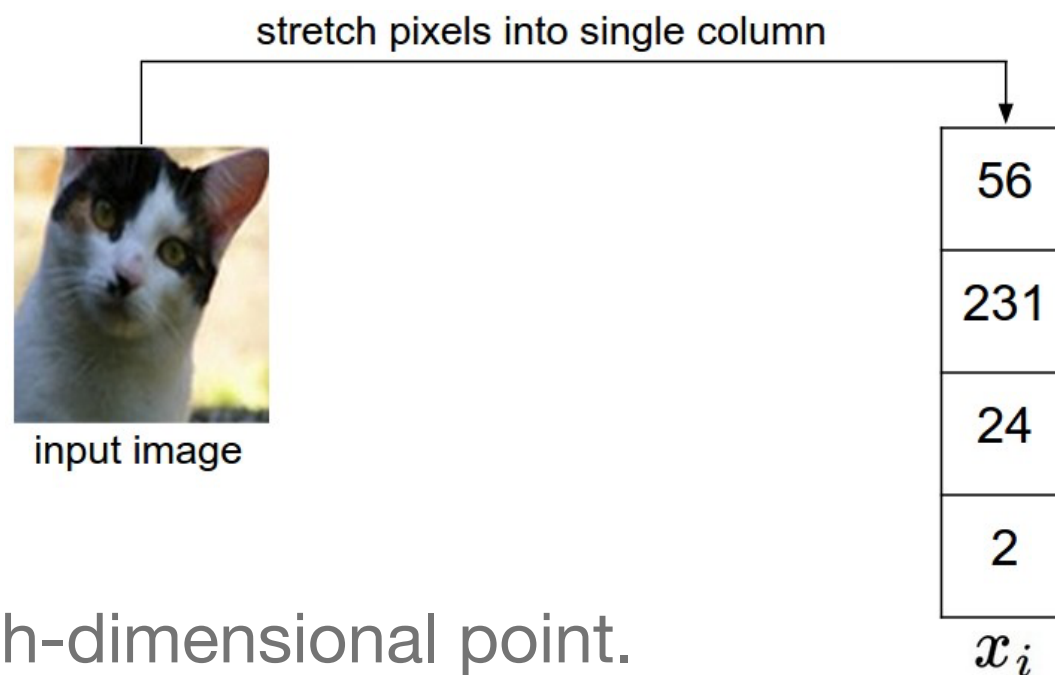
Fundamentals of DL - score function

Score function: map from data to class scores

$$f(x_i, W, b) = Wx_i + b$$

- linear image classifier example

parametric-model. **w**eights and **b**iases.
(parametric: no need to keep all training data)



source: <http://cs231n.github.io>

- image: high-dimensional point.
- score: weighted sum of all pixel values
- one classifier (template) per row

Fundamentals of DL

What about deep nets?



linear classifier

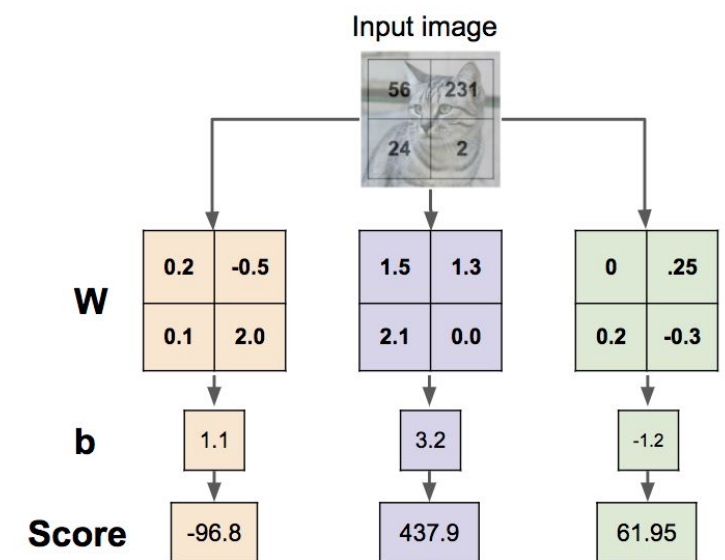
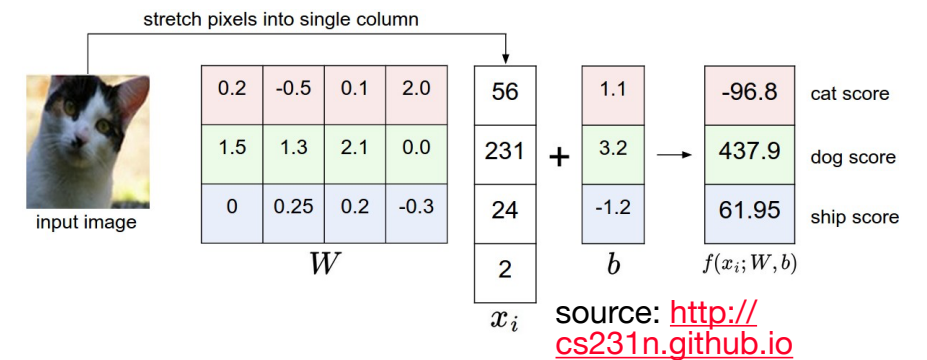


Fundamentals of DL

What about deep nets?



linear classifier

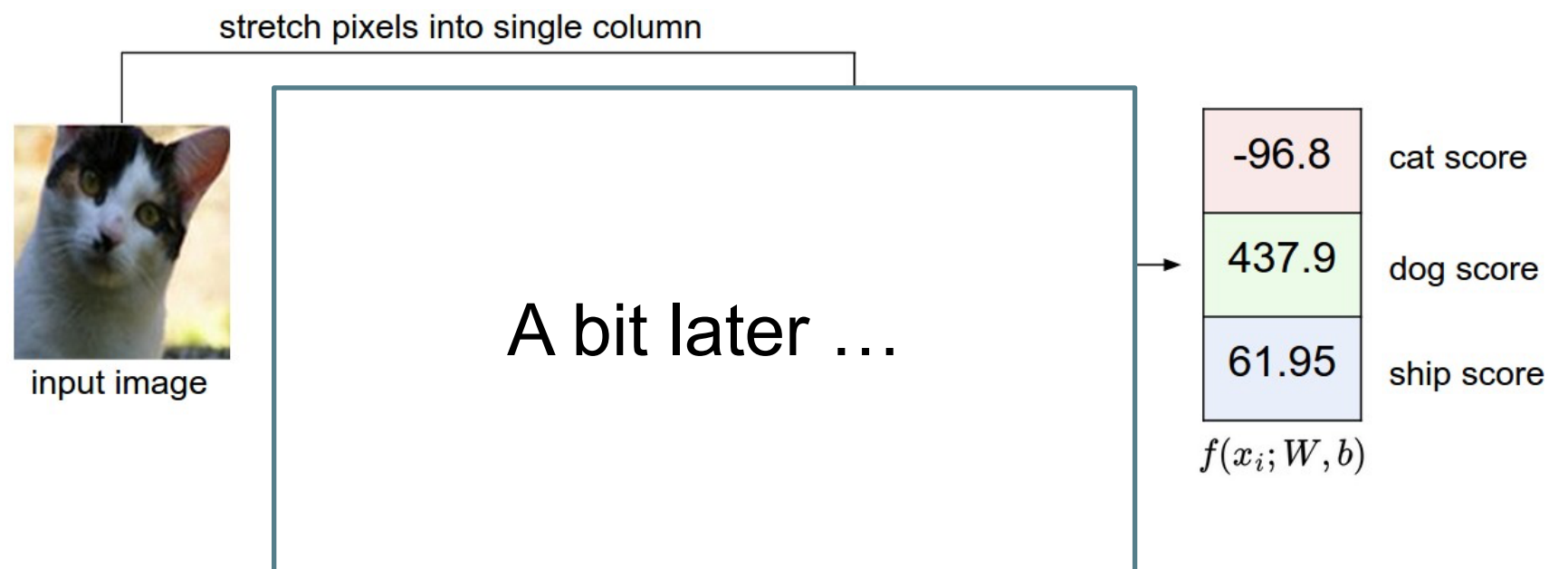


Fundamentals of DL - score function

$$\boxed{W} \cdot \begin{bmatrix} 56 & 231 \\ 24 & 2 \end{bmatrix} = \begin{bmatrix} S_1 \\ S_2 \\ S_3 \end{bmatrix}$$

Score function: map from data to class scores

- Deep nets (CNN in particular) also do this with images but much more complex mapping function and params



- Common practice:

- **preprocessing** (e.g., subtract mean, normalization)

- single multiplication

$$f(x_i, W, b) = Wx_i + b$$

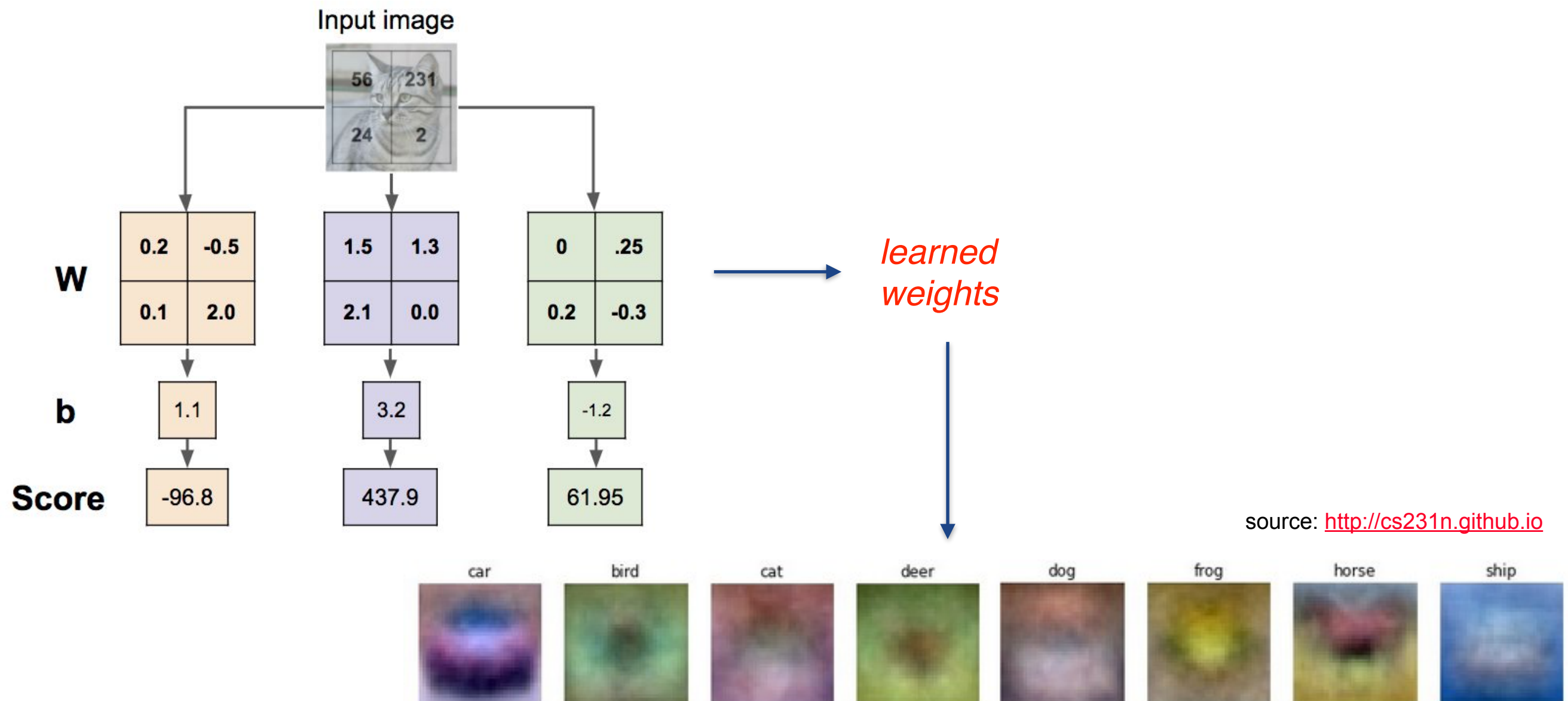
$$f(x_i, W) = Wx_i$$

Fundamentals of DL - score function

$$\mathbf{W} \cdot \begin{bmatrix} 56 & 231 \\ 24 & 2 \end{bmatrix} = \begin{bmatrix} S_1 \\ S_2 \\ S_3 \end{bmatrix}$$

- **Score function: map from data to class scores**

Deep nets (CNN in particular) also do this with images but much more complex mapping function and params (more details in next classes)



Fundamentals of DL

How good are the scores?

Loss [cost | objective] function

Fundamentals of DL - loss

Loss function: *How good are the predictions (scores)?*

$$L = \underbrace{\frac{1}{N} \sum_i L_i}_{\text{data loss}} + \underbrace{\lambda R(W)}_{\text{regularization loss}}$$

Some examples:

- **Multiclass SVM** (one of the possible formulations)

- *wants* correct class to have a score higher than incorrect classes by some **fixed margin**

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

**hinge or
max-margin loss**

the score of every incorrect class - score of actual class

Fundamentals of DL - loss

Loss function: *How good are the predictions (scores)?*

$$L = \underbrace{\frac{1}{N} \sum_i L_i}_{\text{data loss}} + \underbrace{\lambda R(W)}_{\text{regularization loss}}$$

Some examples:

- **Cross-entropy/Softmax loss:**

- equivalent to “difference” between perfect and actual distribution

$$H(p, q) = - \sum_x p(x) \log q(x) \quad \longrightarrow \quad L_i = - \log \left(\frac{e^{f_{y_i}}}{\sum_j e^{f_j}} \right)$$

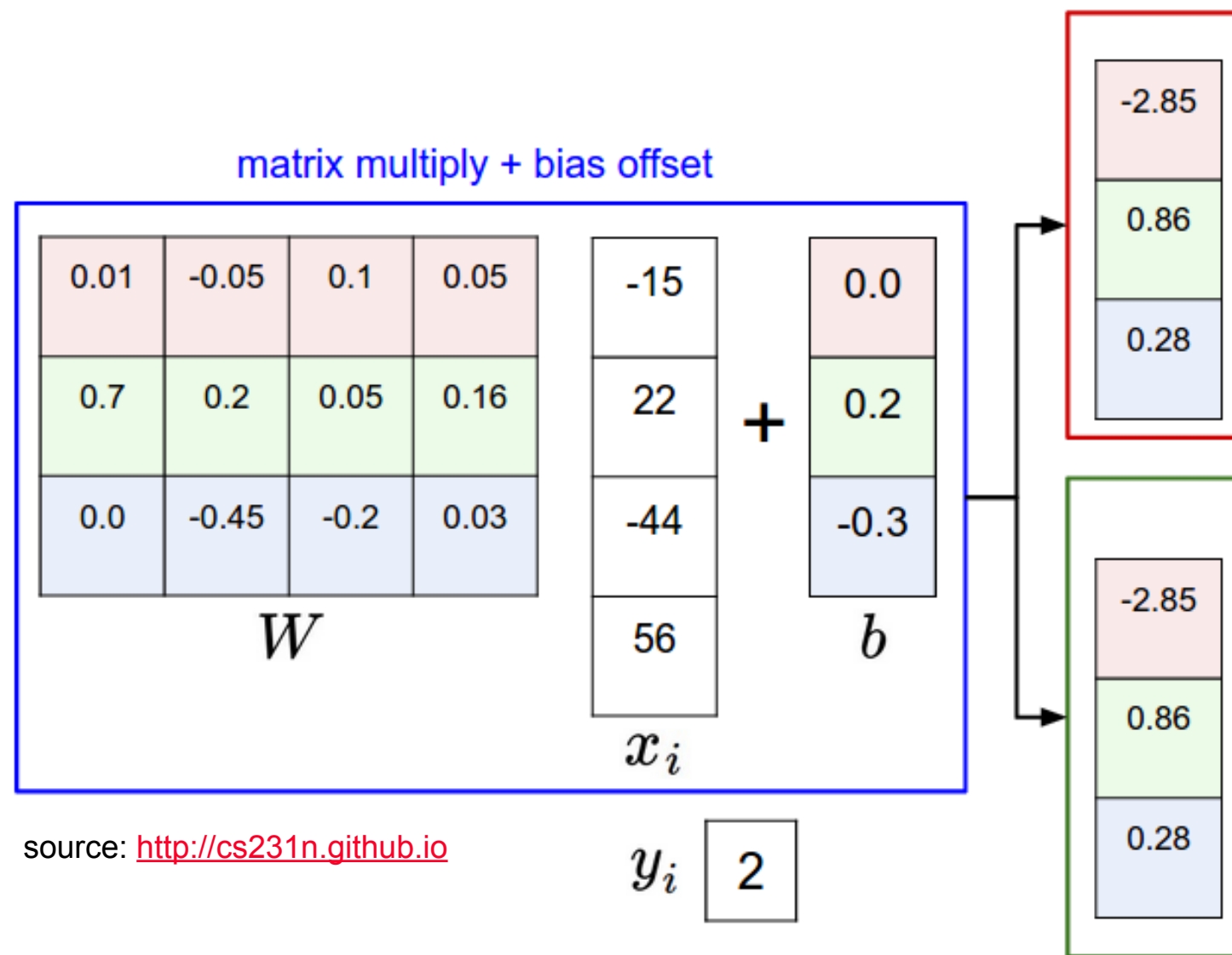
Cross entropy between distributions p, q
Ideally only the actual label is true:
 $\mathbf{p} = [0, \dots, 1, \dots, 0]$

Softmax Function: score of actual class normalised

Fundamentals of DL - loss

$$L = \underbrace{\frac{1}{N} \sum_i L_i}_{\text{data loss}} + \underbrace{\lambda R(W)}_{\text{regularization loss}}$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$



Try to compute the two loss functions ...

$$L_i = -\log\left(\frac{e^{f_{y_i}}}{\sum_j e^{f_j}}\right)$$

source: <http://cs231n.github.io>

Fundamentals of DL - loss

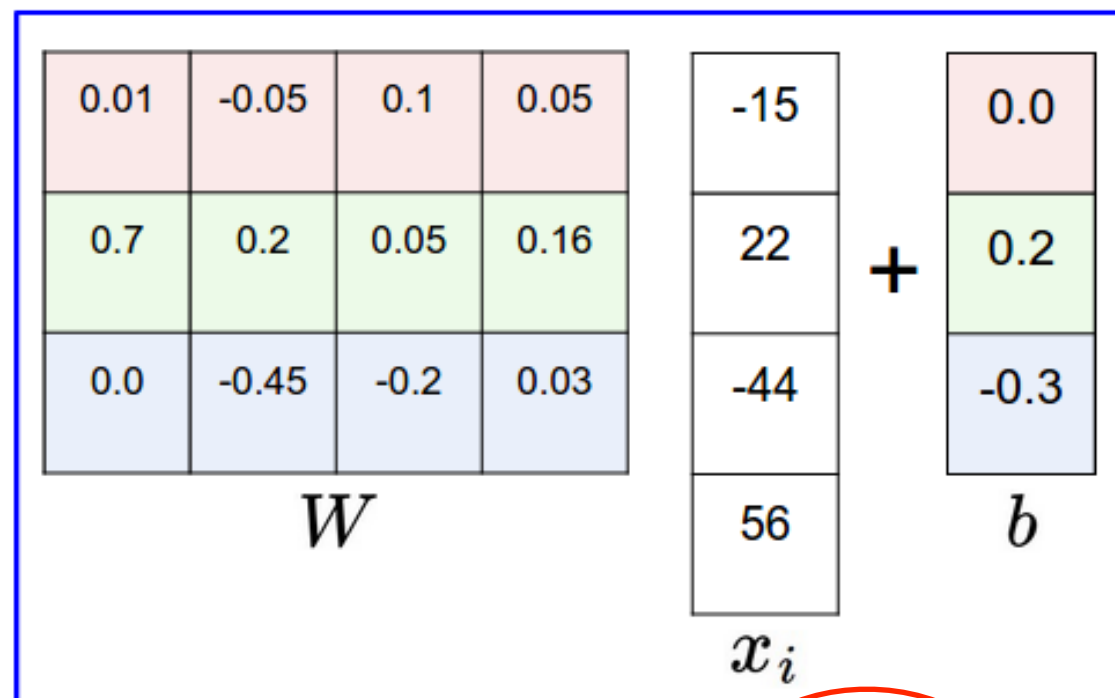
$$L = \underbrace{\frac{1}{N} \sum_i L_i}_{\text{data loss}} + \underbrace{\lambda R(W)}_{\text{regularization loss}}$$

SVM vs Soft-max loss: comparable but different *meaning* of output values

Softmax interprets scores as probabilities

Q1: Can we know the min/max possible L_i ?
Q2: What loss can we expect at initialisation?

matrix multiply + bias offset

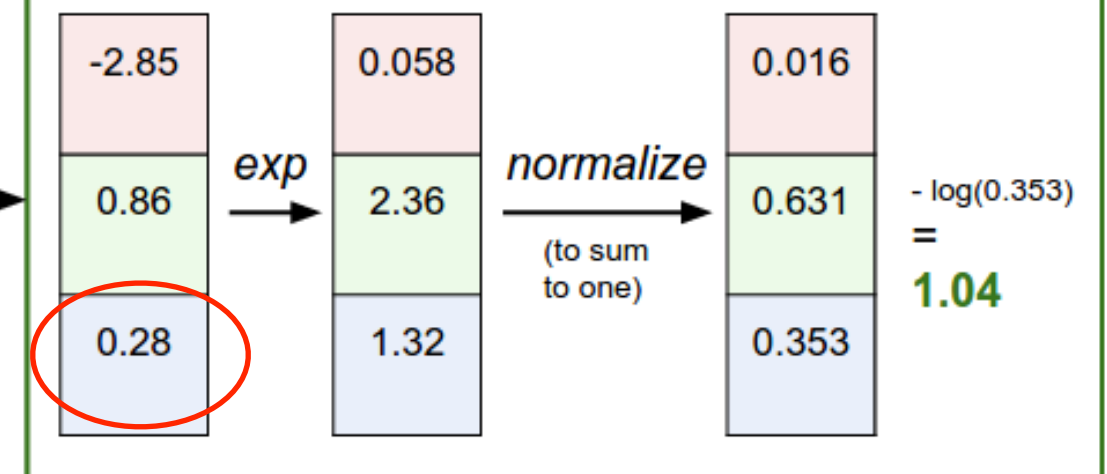


source: <http://cs231n.github.io>

y_i 2

$$L_i = -\log\left(\frac{e^{f_{y_i}}}{\sum_j e^{f_j}}\right)$$

cross-entropy loss (Softmax)



Fundamentals of DL - loss

How good are the scores?

Many options for loss functions

—> **critical to choose a right one**

Data Type	Domain	Distribution	Use
univariate, continuous, unbounded	$y \in \mathbb{R}$	univariate normal	regression
univariate, continuous, unbounded	$y \in \mathbb{R}$	Laplace or t-distribution	robust regression
univariate, continuous, unbounded	$y \in \mathbb{R}$	mixture of Gaussians	multimodal regression
univariate, continuous, bounded below	$y \in \mathbb{R}^+$	exponential or gamma	predicting magnitude
univariate, continuous, bounded	$y \in [0, 1]$	beta	predicting proportions
multivariate, continuous, unbounded	$\mathbf{y} \in \mathbb{R}^K$	multivariate normal	multivariate regression
univariate, continuous, circular	$y \in (-\pi, \pi]$	von Mises	predicting direction
univariate, discrete, binary	$y \in \{0, 1\}$	Bernoulli	binary classification
univariate, discrete, bounded	$y \in \{1, 2, \dots, K\}$	categorical	multiclass classification
univariate, discrete, bounded below	$y \in [0, 1, 2, 3, \dots]$	Poisson	predicting event counts
multivariate, discrete, permutation	$\mathbf{y} \in \text{Perm}[1, 2, \dots, K]$	Plackett-Luce	ranking

Figure 5.11 Distributions for loss functions for different prediction types.

Fundamentals of DL - loss

Loss function: *How good are the predictions (scores)?*

$$L = \underbrace{\frac{1}{N} \sum_i L_i}_{\text{data loss}} + \underbrace{\lambda R(W)}_{\text{regularization loss}}$$

“Regularization is any modification we make to a learning algorithm that is intended to reduce its generalization error but not its training error.”

- **Regularization.**

(Goodfellow 2016)

- improve generalization (less overfitting). L2, L1, Dropout, batch-norm, ...
- models preferences: e.g., L2, discourages large values

$$\lambda \sum_k \sum_l W_{k,l}^2$$

regularization **strength**
(hyperparameter)

$$\begin{aligned} x &= [1, 1, 1, 1] \\ w_1 &= [1, 0, 0, 0] \\ w_2 &= [0.25, 0.25, 0.25, 0.25] \end{aligned}$$

$$w_1^T x = w_2^T x = 1$$

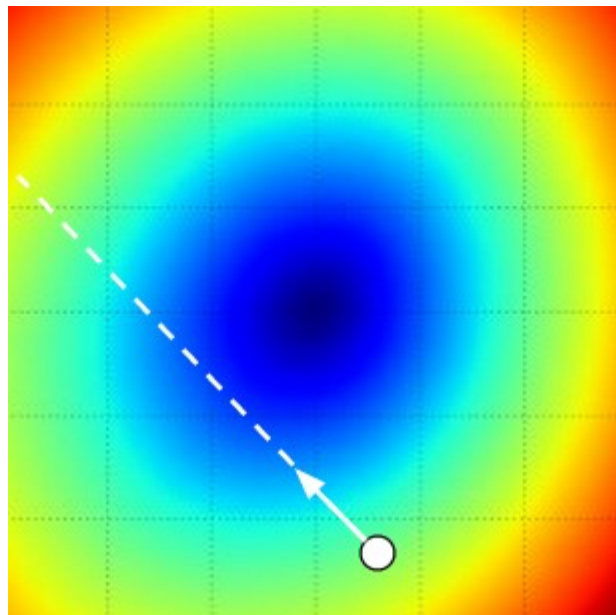
What's best with L2?
and with L1?

Fundamentals of DL

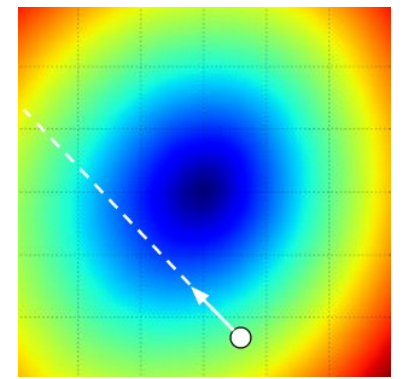
We can measure how good a set of parameters (W) is

How do we find the best W ?

Optimisation (training)



Fundamentals of DL - Optimisation



Optimisation —> best way to find the parameters.

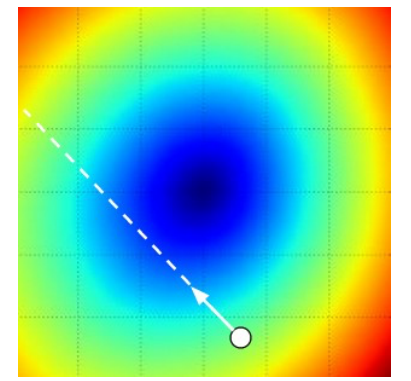
*We know how to measure how good a set of parameters is.
Start with random weights. Iteratively refine to get lower loss.*

Some options ...

- Random search or “local” search: not *ideal* for DL. Used in other problems
- Follow the “gradient”: vector of derivatives for each dimension of input space
 - compute **gradient of loss function with respect to model weights W**
 - step size (*learning rate*) hyperparameter

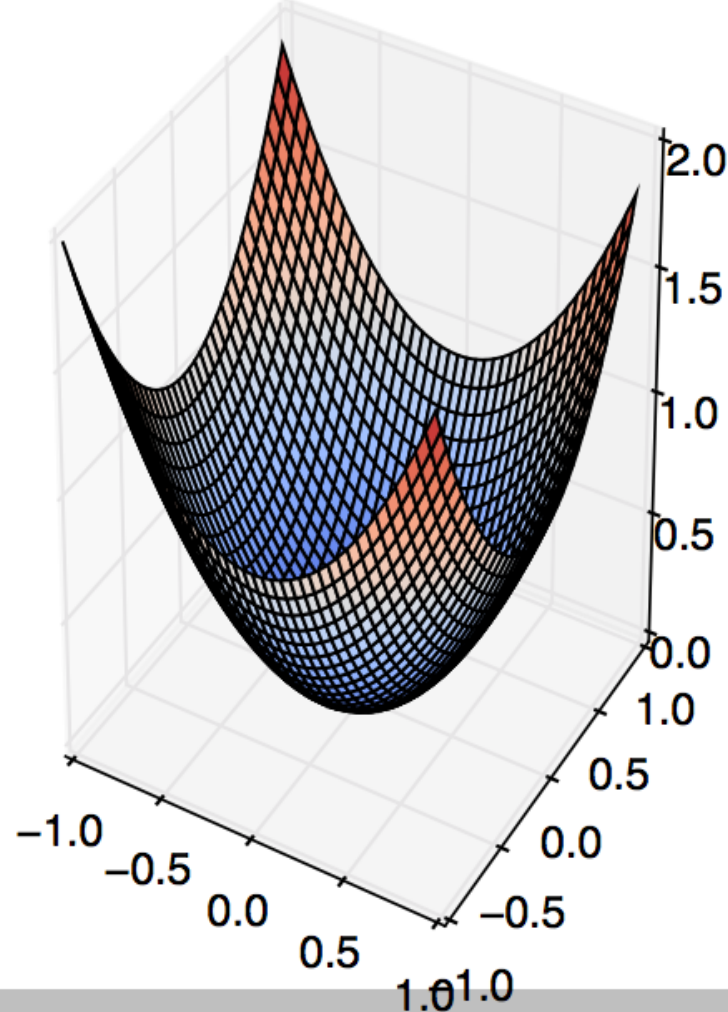
$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(x_i, y_i, W) + \lambda R(W)$$
$$\nabla_W L(W) = \frac{1}{N} \sum_{i=1}^N \nabla_W L_i(x_i, y_i, W) + \lambda \nabla_W R(W)$$

Fundamentals of DL - Optimisation



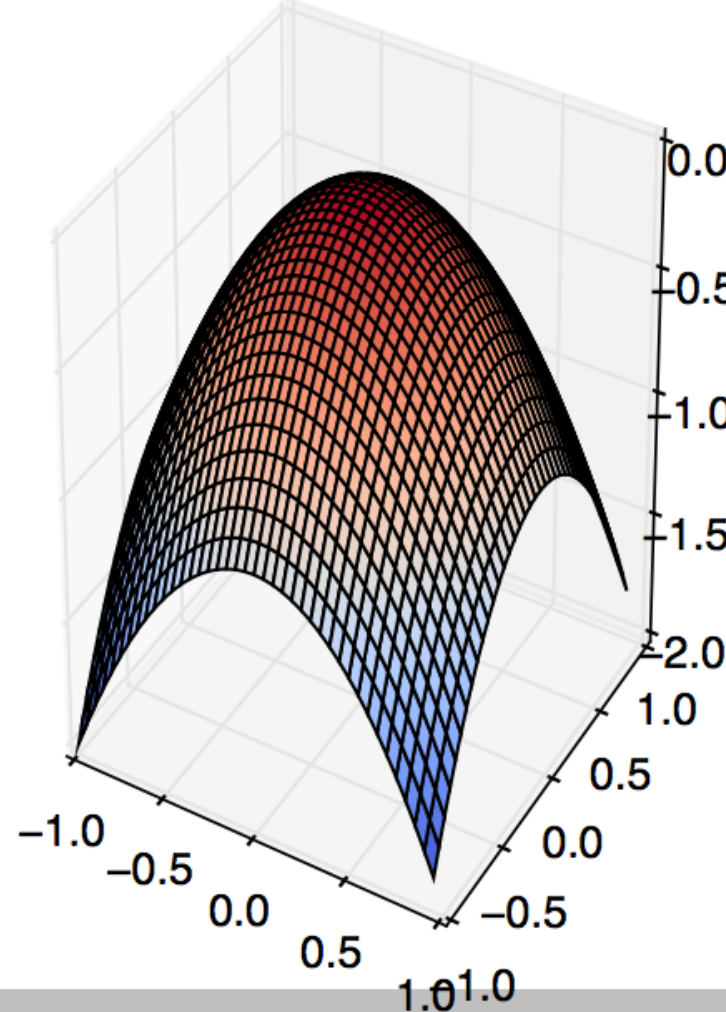
Optimisation. Critical points: Zero gradient, and Hessian with...

Minimum



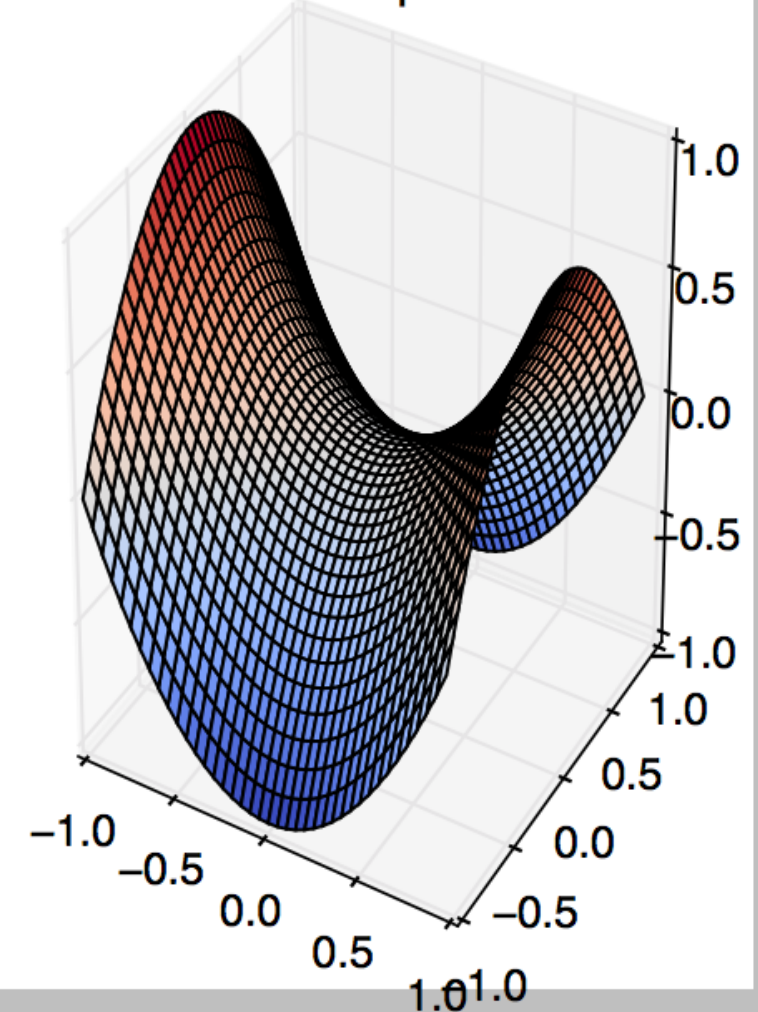
All positive eigenvalues

Maximum



All negative eigenvalues

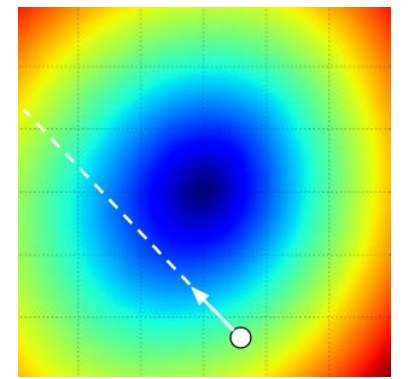
Saddle point



Some positive
and some negative

(Goodfellow 2015)

Fundamentals of DL - Optimisation



- **Optimisation** - Gradient Descent (**GD**) & **Deep Learning**.

- Ideally, find the exact minimum. Actually, not feasible. N is very large (#samples). W has too many params (1K-1000K)

$$\nabla_W L(W) = \frac{1}{N} \sum_{i=1}^N \nabla_W L_i(x_i, y_i, W) + \lambda \nabla_W R(W)$$

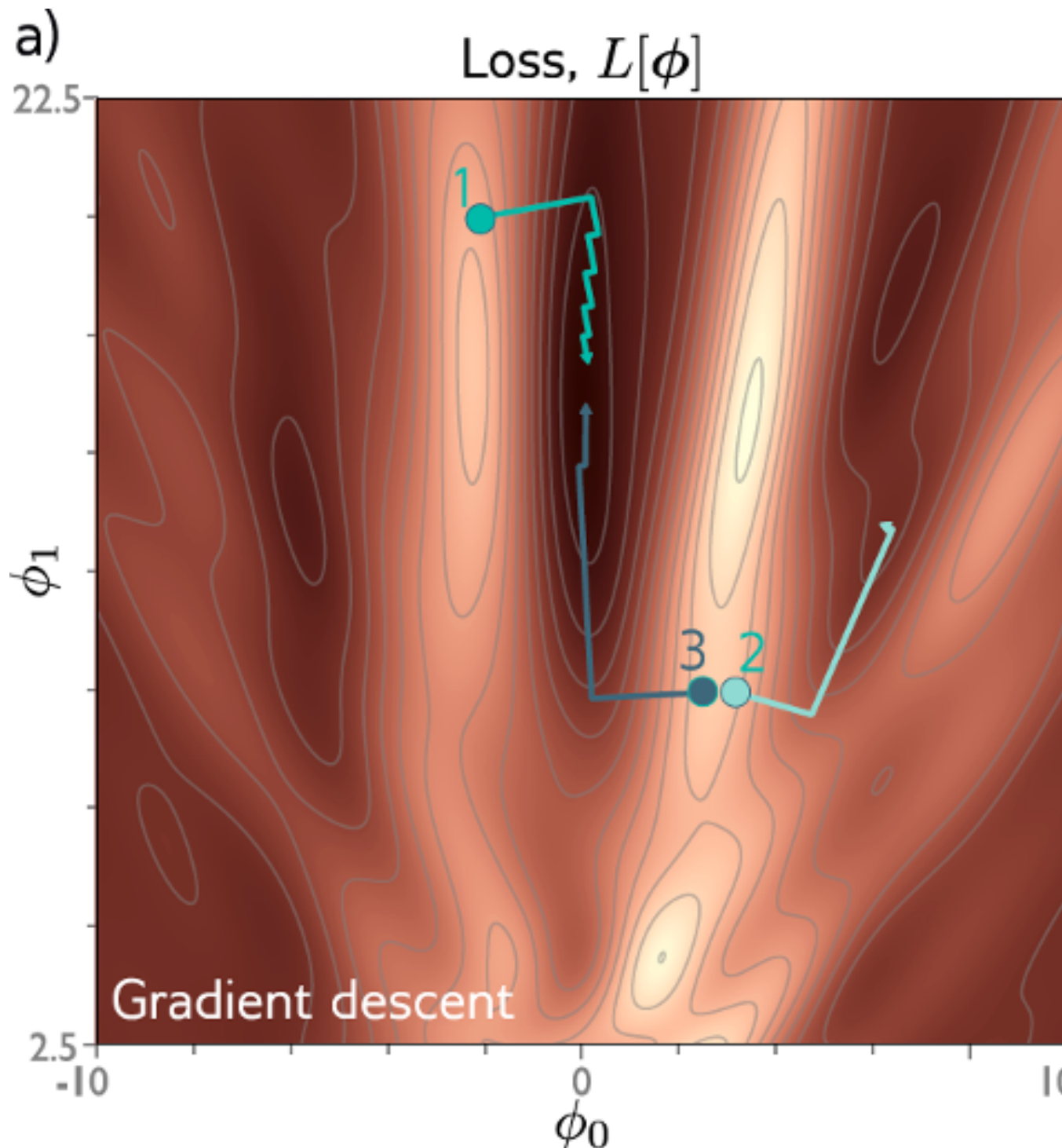
- Saddle points are a problem. Many methods get stuck there. Stochastic GD has better chances.

$$\theta = \theta - \alpha \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} J_i(\theta) \quad \textbf{GD: for all samples (or Batch GD)}$$

$$\theta = \theta - \alpha \nabla_{\theta} J_i(\theta) \quad \textbf{SGD: for one sample (or a few)}$$

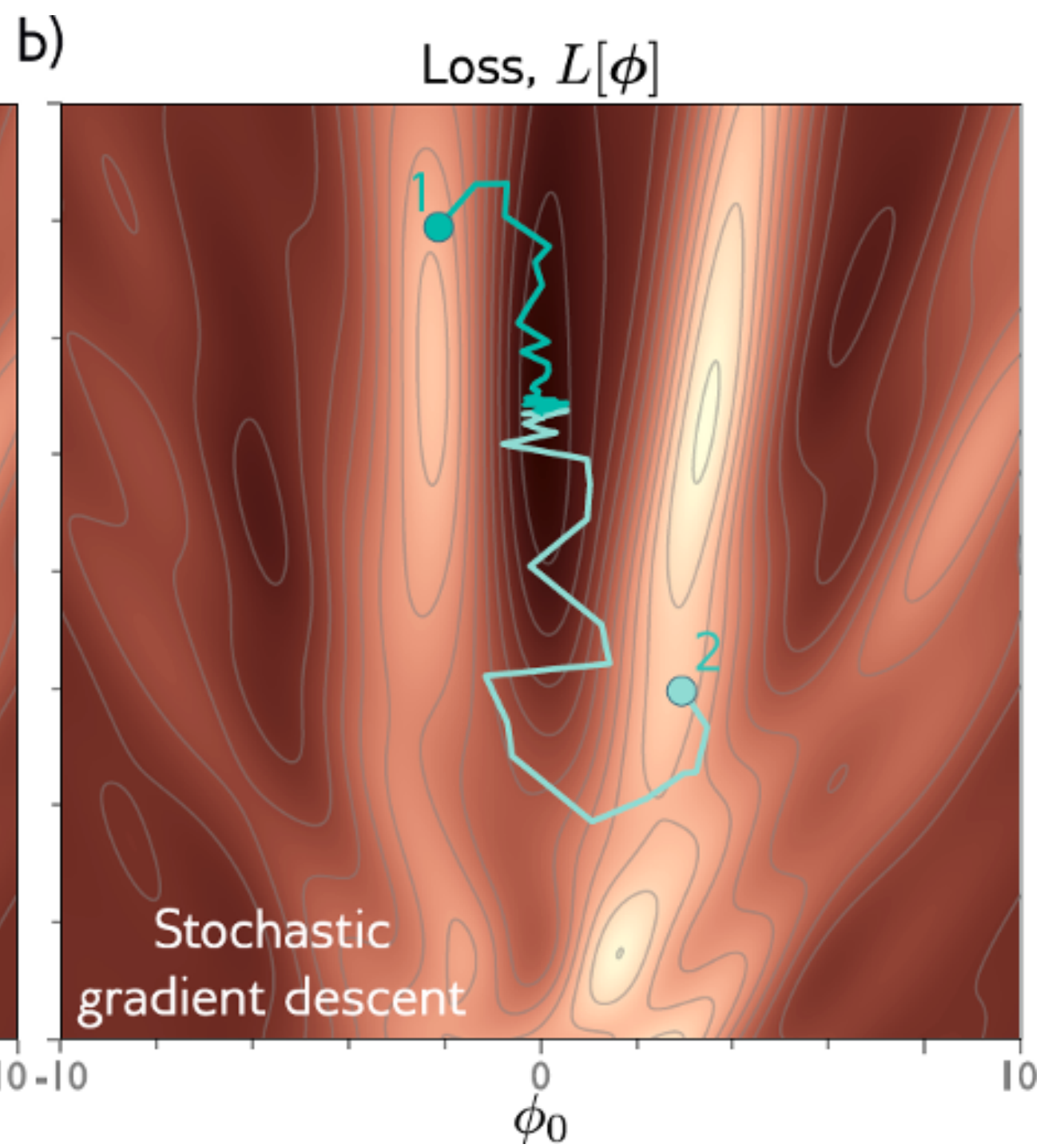
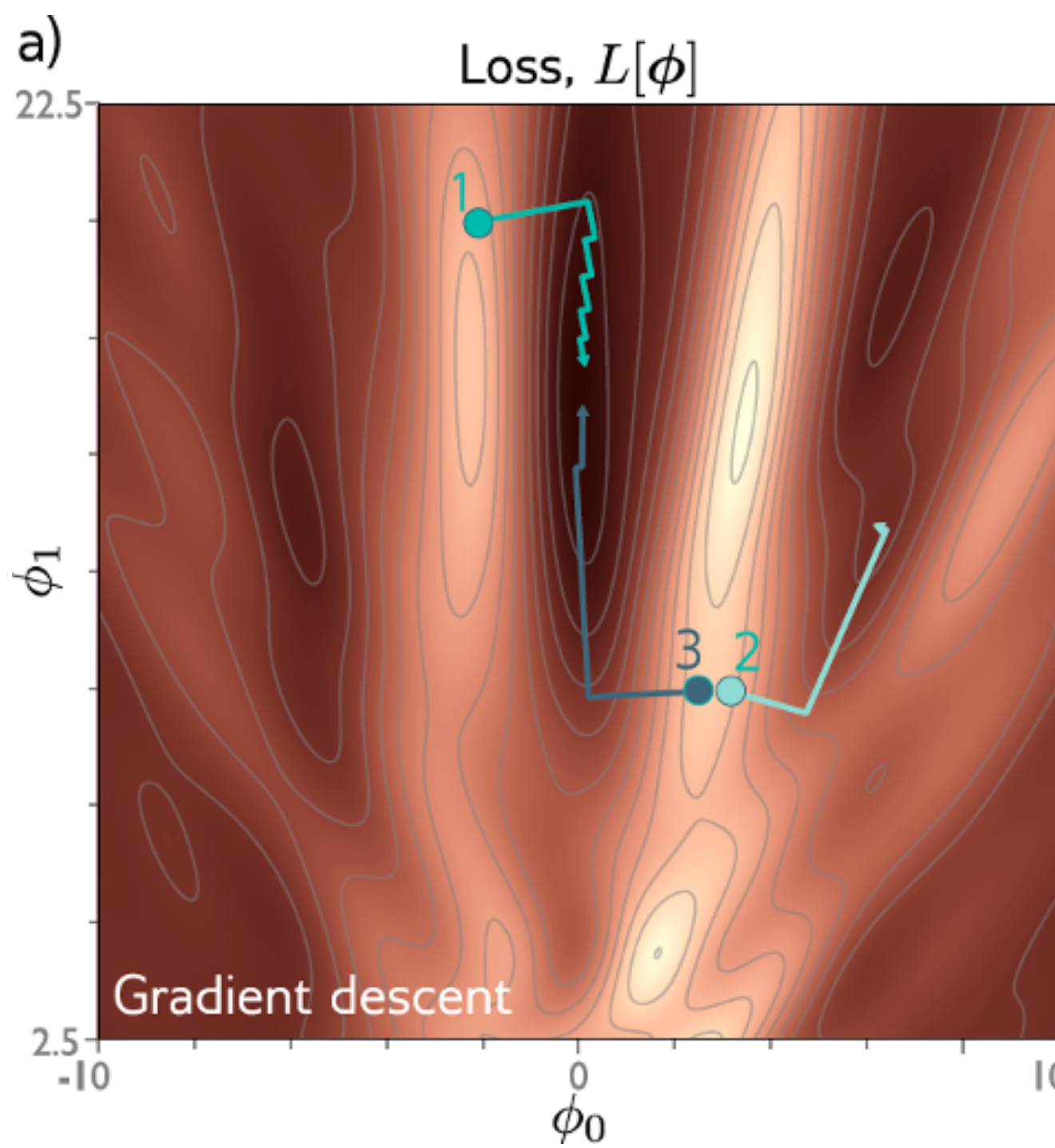
- **Stochastic GD (SGD)**: start “improving” from the first sample processed. One step per sample (or per **mini-batch**: 32, 64, 128, 256, ... No more that you can fit in your CPU/GPU memory!).
- Most common training strategy in DL: SGD + Backpropagation (later)

Visualising GD vs SGD



IDEA: add noise

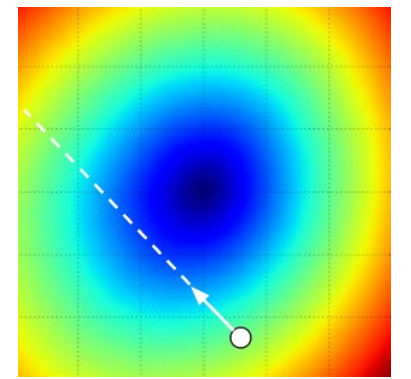
- **Stochastic gradient descent**
- Compute gradient based on only a subset of points – a **mini-batch**
- Work through dataset sampling without replacement
- One pass through the data is called an **epoch**



Properties of SGD

- Can escape from local minima
- Adds noise, but still sensible updates as based on part of data
- Uses all data equally
- Less computationally expensive
- Seems to find better solutions
- Doesn't converge in traditional sense
- Learning rate schedule – decrease learning rate over time

Fundamentals of DL - Optimisation

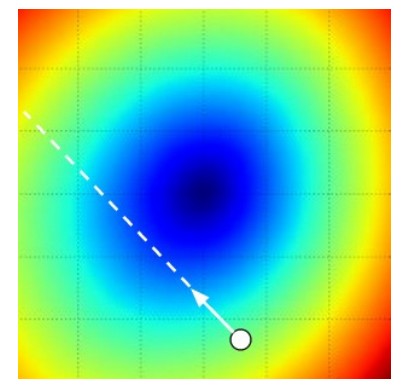


- **Optimisation** - SGD - Still many challenges for the *vainilla* version

   ?	BGD	SGD
1. redundant computations for similar examples		
2. faster		
3. high variance and high fluctuations during training		
4. converges to the minimum of the “basin” where it starts		
5. needs to shuffle training data		
6. suitable for online or incremental learning		

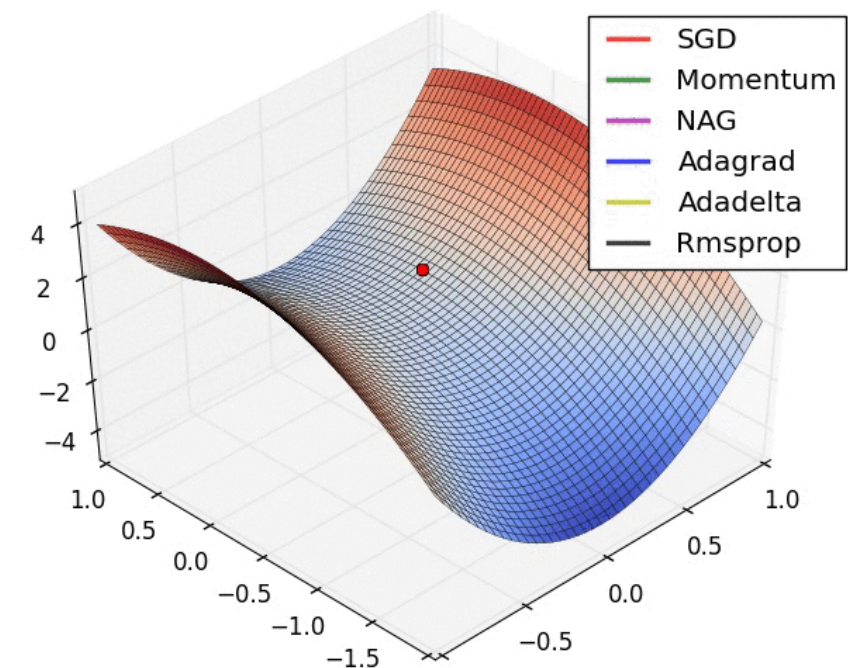
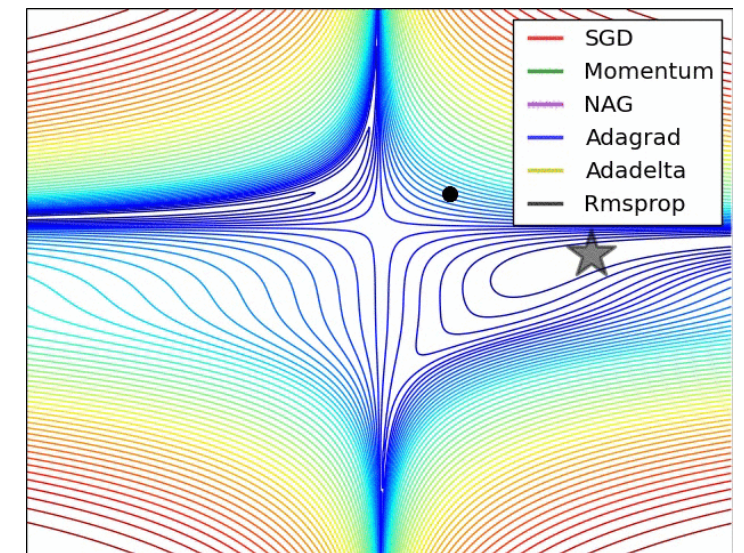
What's true for which variation?

Fundamentals of DL - Optimisation

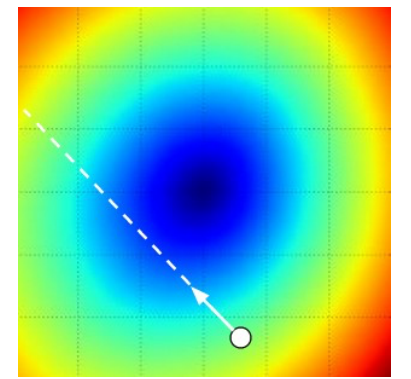


- **Optimisation** - SGD -
Many optimization algorithms with different properties

- Loss surface contours and time evolution of different algorithms (behaviours, speeds, stuck-points, ...)
- **Adaptive learning-rate** - LR per param, adapted at each step) - best compromise in saddle point problems (where Vanilla SGD gets stuck).
- **Weight-decay** - reduce weight values at each step. Can be equivalent to L2 regularization term. In adaptive approaches use “*decoupled*” version (as in **AdamW**)



Fundamentals of DL - Optimisation



- **Optimisation** - SGD - Many optimization algorithms with different properties

- Momentum

- Average

- AdaGrad

- RMSProp

- Adam / AdamW

- Nesterov Accelerated gradient (NAG)

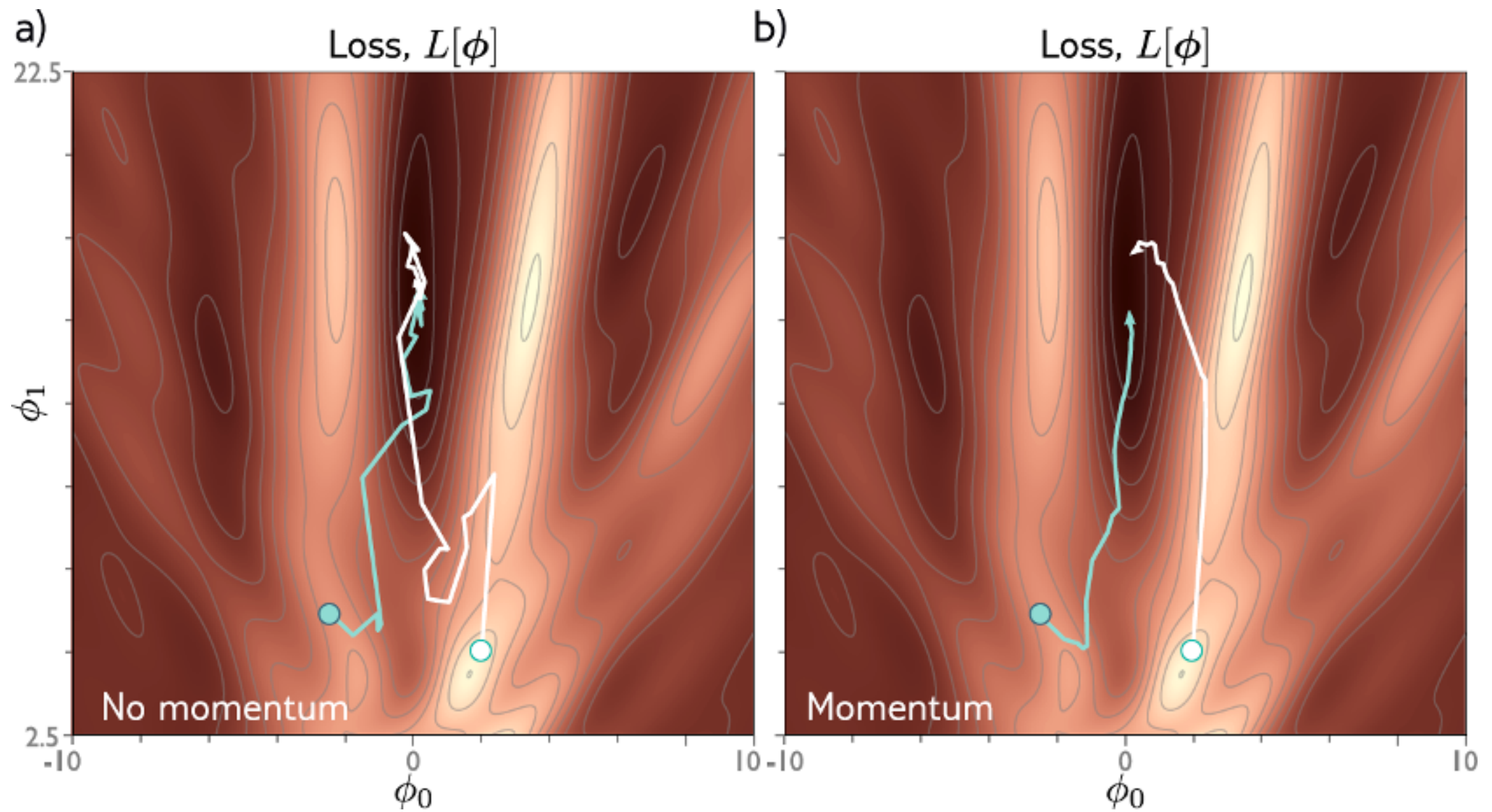
Many available optimizers on
standard libraries

Momentum

- Weighted sum of this gradient and previous gradient

$$\mathbf{m}_{t+1} \leftarrow \beta \cdot \mathbf{m}_t + (1 - \beta) \sum_{i \in \mathcal{B}_t} \frac{\partial \ell_i[\phi_t]}{\partial \phi}$$

$$\phi_{t+1} \leftarrow \phi_t - \alpha \cdot \mathbf{m}_{t+1}$$



Adam

- Gradient normalisation + Momentum
- Why normalise the gradient?

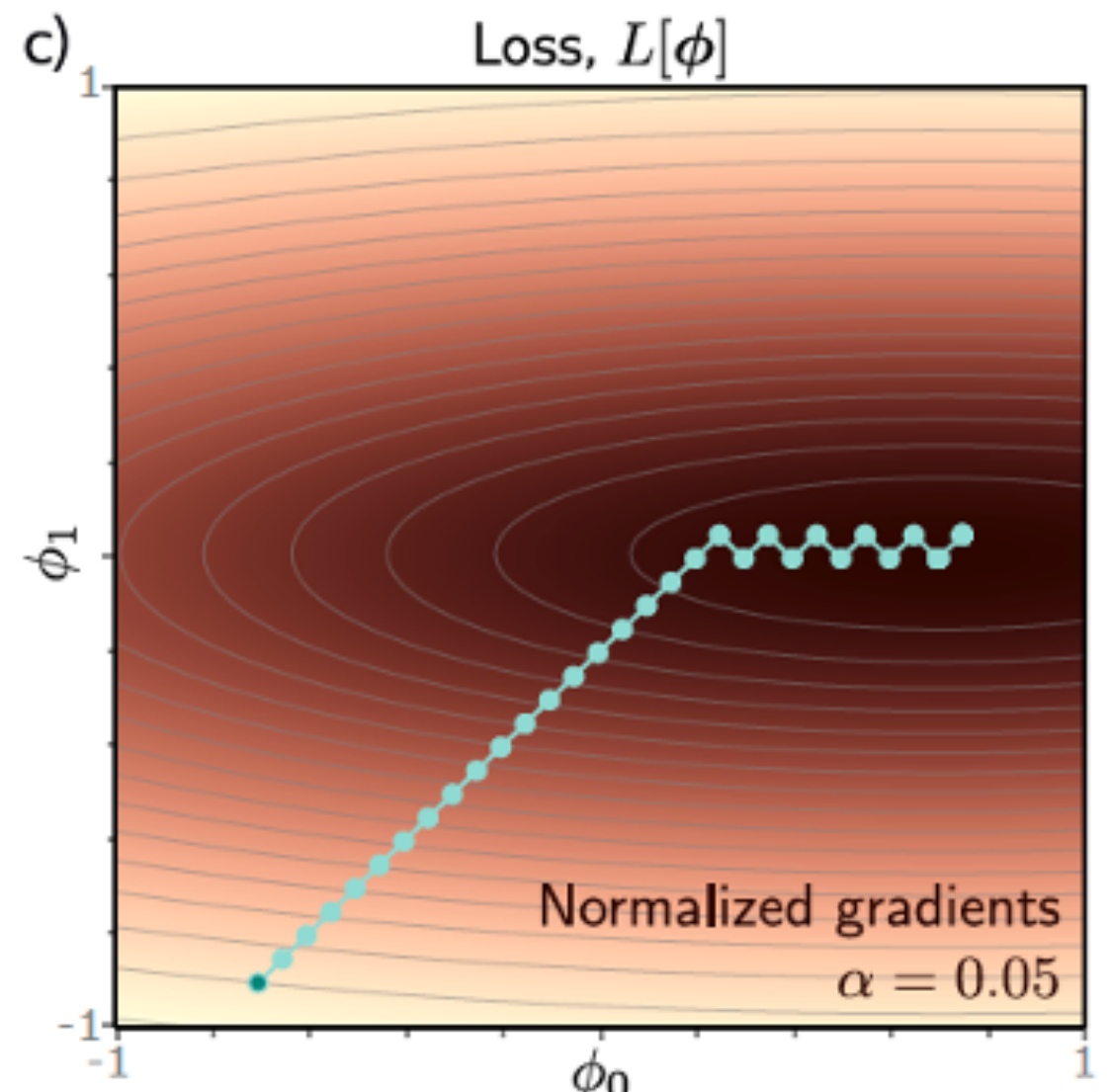
Adam

- Gradient normalisation + Momentum
- Why normalise the gradient?
- How?

$$\mathbf{m}_{t+1} \leftarrow \frac{\partial L[\phi_t]}{\partial \phi}$$

$$\mathbf{v}_{t+1} \leftarrow \frac{\partial L[\phi_t]^2}{\partial \phi}$$

$$\phi_{t+1} \leftarrow \phi_t - \alpha \cdot \frac{\mathbf{m}_{t+1}}{\sqrt{\mathbf{v}_{t+1}} + \epsilon}$$

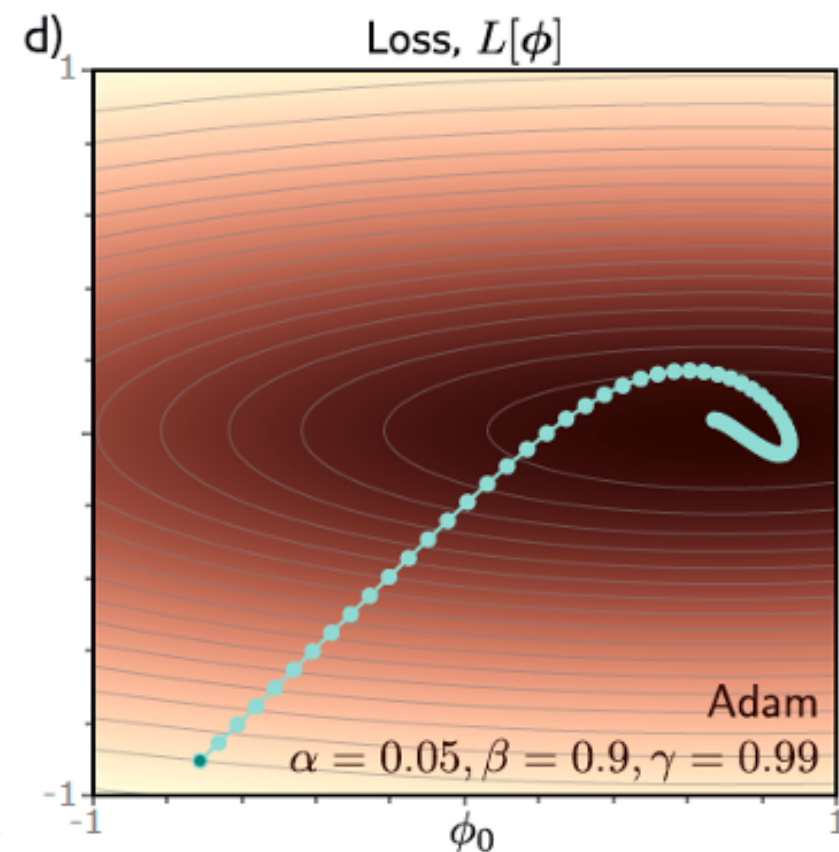
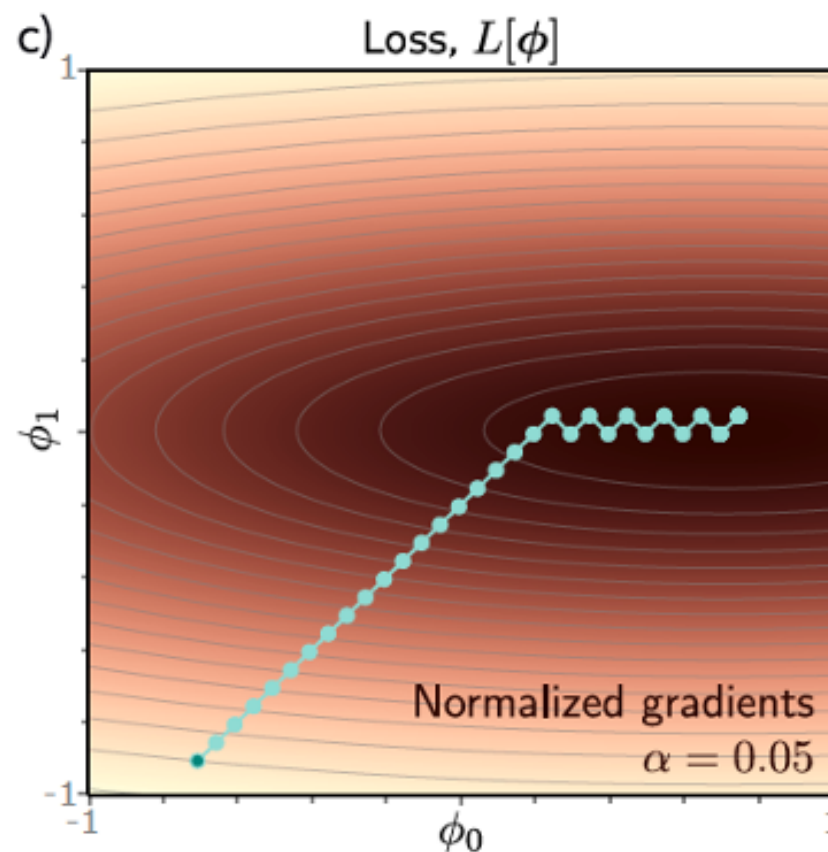


- Gradient normalisation + Momentum

- Compute momentum
$$\mathbf{m}_{t+1} \leftarrow \beta \cdot \mathbf{m}_t + (1 - \beta) \frac{\partial L[\phi_t]}{\partial \phi}$$
$$\mathbf{v}_{t+1} \leftarrow \gamma \cdot \mathbf{v}_t + (1 - \gamma) \left(\frac{\partial L[\phi_t]}{\partial \phi} \right)^2$$

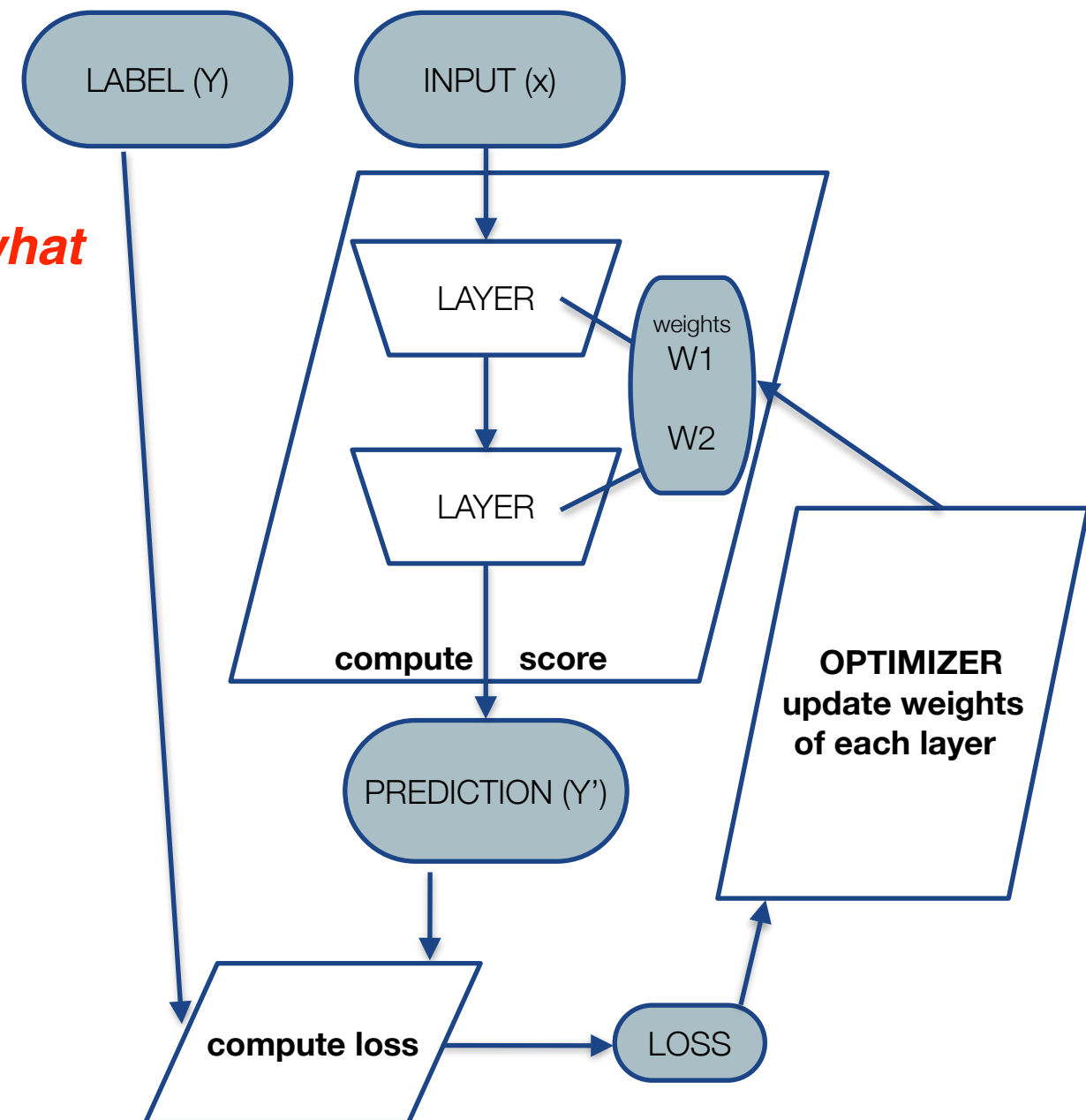
- Moderate at the start

$$\tilde{\mathbf{m}}_{t+1} \leftarrow \frac{\mathbf{m}_{t+1}}{1 - \beta^{t+1}}$$
$$\tilde{\mathbf{v}}_{t+1} \leftarrow \frac{\mathbf{v}_{t+1}}{1 - \gamma^{t+1}}$$



Fundamentals of DL - a few key concepts

- **Score** function $\rightarrow$ map from data to class scores.
Weights and biases (w, b) ← This is **what** we learn
- **Loss** function $\rightarrow$ how good are the predictions (scores).
SVM, SoftMax, ... ↗ This is **how** we learn
- **Optimisation** $\rightarrow$ best way to find the parameters (w, b): **Train**



Fundamentals of DL - a few key concepts

- We have added more hyper parameters
 - Choice of learning algorithm
 - Learning rate
 - Momentum
- What's next?